Credit Card Fraud Detection:
A Critical Reproduction Study

Data Science in Cyber, Dr. Uri Itai | Individual Final Project

Source reviewed: V. Valkov, *"Credit Card Fraud Detection using Autoencoders in Keras"* (curiousily.com; GitHub, ~588 stars). Dataset: ULB credit-card fraud (Kaggle mlg-ulb, OpenML 42175).

1. Summary of the Source

Problem. The task is to identify fraudulent credit-card transactions in the ULB dataset, which contains 284,807 transactions made by European cardholders over two days. Only 492 of them (0.17%) are fraudulent. The predictors are 28 anonymised PCA components (V1 to V28) together with the transaction time and amount.

Why it matters. Card fraud accounts for very large annual losses, and a practical detector has to find the small number of fraudulent transactions without blocking too many legitimate customers. The severe class imbalance is the main technical difficulty and is also the reason a naive evaluation can be misleading.

Proposed solution. The author trains a semi-supervised autoencoder with layer sizes 29-14-7-7-29 on legitimate transactions only. The network learns to reconstruct normal behaviour, and at inference a transaction whose reconstruction error exceeds a fixed threshold (the author uses 2.9) is labelled as fraud. The reported result is a ROC-AUC of roughly 0.95.

Data and methodology. The author uses an 80/20 train/test split, keeps only the legitimate rows for training, standardises the amount, and trains for 100 epochs with the Adam optimiser and a mean-squared-error loss. Results are summarised with a ROC curve, a precision-recall curve, and a confusion matrix at the 2.9 threshold.

2. Critical Evaluation of the Author's Claims

The central claim is that reconstruction error produces a strong fraud detector, with a ROC-AUC near 0.95. This claim reproduces. Our faithful 100-epoch port yields a ROC-AUC of 0.961. The problem is that this figure is not good evidence that the detector is useful, for three reasons.

Test-set information leaks into the reported score in three places. First, the amount is standardised with a scaler fitted on the full dataset before the split, so training statistics include the test set. Second, the test set is passed as the validation set, and the checkpoint callback keeps the model with the lowest validation loss, which means the saved model is selected on test data. Third, the decision threshold of 2.9 is read off reconstruction-error plots that are drawn on the labelled test set. Every one of these choices lets test information influence the reported number, so the result is optimistic by construction.

ROC-AUC is not an appropriate headline at 0.17% prevalence. When 99.83% of rows are negative, ROC-AUC is dominated by the many easily ranked negatives and can look strong even when the operating point is weak. The threshold-free precision-recall AUC, which is the more honest summary for a rare positive class, is only 0.231 for the same model. At the 2.9 threshold the detector produces 1,289 false alarms in order to catch 79 of 98 frauds, a precision of 5.77%.

The accuracy metric compiled into the network is not meaningful. An autoencoder trained with mean-squared error is a regression model, so the reported per-element accuracy measures reconstruction agreement rather than detection quality and carries no information about fraud detection.

Are the conclusions justified? They are not. As Section 5 shows, once the leaks are removed, ordinary supervised classifiers achieve a higher PR-AUC than the autoencoder. The author's numbers can be reproduced, but the interpretation that a high ROC-AUC implies a good detector overstates what the experiment actually demonstrates.

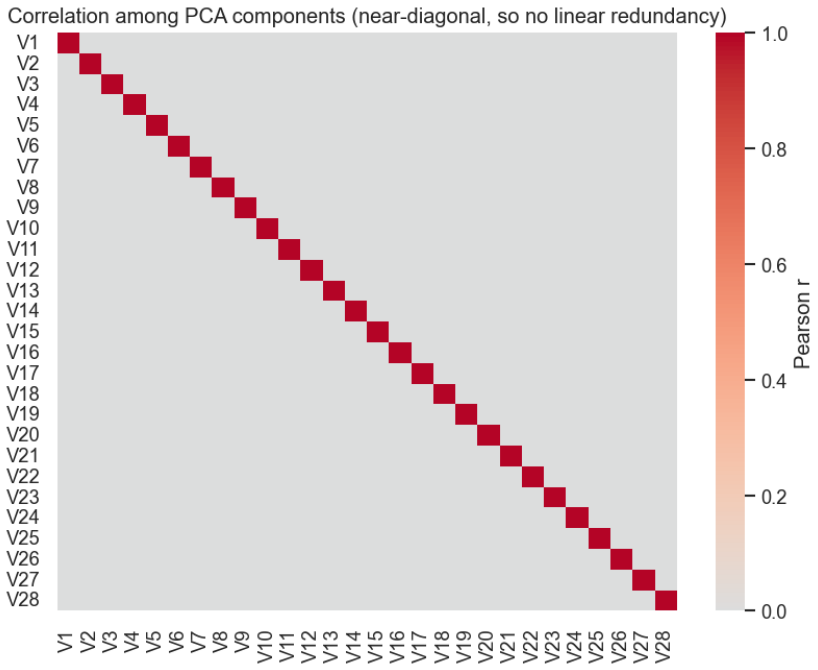
3. Feature Engineering Analysis

What the source does. The feature engineering is minimal. The author removes the time column and standardises the amount, and uses the supplied PCA components unchanged.

Redundancy. Because V1 to V28 are PCA outputs, they are linearly uncorrelated by construction. We confirm this: the largest absolute off-diagonal correlation among them is 0.000, and the correlation matrix is visually diagonal. There is therefore no linear redundancy to remove, and no reason to apply a further dimensionality reduction such as PCA, since the data is already an orthogonal reduced representation. Any remaining redundancy would be non-linear and is better handled by the models than by manual feature removal. To detect redundancy in a general dataset we would inspect a correlation matrix and the variance inflation factor, and drop or combine highly collinear columns.

Our engineering. We make two changes. We reconstruct an hour-of-day feature from the time column, which the exploratory analysis shows is informative because the fraud rate is higher during quiet night hours. We also scale the amount with a RobustScaler, which centres on the median and scales by the interquartile range, because the amount is heavy-tailed with extreme outliers that would distort a standard scaler. The scaler is fitted on the training split only.

Additional features that could help. With the raw transaction records (not available here because of anonymisation) the most valuable additions would be behavioural aggregates per card: the number of transactions and total amount in the last hour and day, the time since the previous transaction, the deviation of the current amount from the cardholder's usual spending, and simple merchant or country risk indicators. These velocity and profile features are known to be strong signals in production fraud systems.



4. Reproducibility Analysis

Does the code run? Not as published. The notebook was written for Keras 1.x and TensorFlow 1.1 in 2017, using imports such as `from keras.models import Model`, an `.h5` checkpoint, and a deprecated `pd.value_counts` call. On a current TensorFlow 2.20 environment it fails at import. We ported it to the `tensorflow.keras` API and a `.keras` checkpoint without changing any modelling choice.

Are the required files and data available? The repository provides the code and a pre-trained model but not the dataset, and the Kaggle download requires an account. We restored a self-contained workflow by loading the same data from OpenML (id 42175). We note that the more common OpenML mirror (id 1597)

drops the time column, which the temporal analysis needs.

Are there hidden preprocessing steps? In effect, yes. The scaler fitted on all data, the use of the test set for model selection, and the manually chosen threshold are easy to overlook when reading the code, yet each one changes the reported result. Overall the work is reproducible only after a non-trivial repair. A fixed random seed and a scripted data loader, as used in this project, are what make the analysis genuinely repeatable.

5. Experimental Results

We evaluate five configurations on the same held-out test set: the original autoencoder at its 2.9 threshold; a corrected autoencoder that uses train-only scaling, early stopping on a validation split, and a threshold selected on validation by F2; and three baselines, namely Logistic Regression and Random Forest (both with balanced class weights) and an Isolation Forest. For every fair model the decision threshold is chosen on the validation set and the test set is used once, for the final measurement only.

5.1 Evaluation metrics

We report the metrics below. For a fraud problem a false positive is a legitimate transaction that gets blocked, which causes customer friction and manual review cost, while a false negative is a fraud that is missed, which causes a direct financial loss and a security failure. Because a missed fraud is generally more costly than a false alarm, we favour recall through the F2 score and select thresholds accordingly.

• **Precision** = TP / (TP + FP). Share of flagged transactions that are truly fraud; low precision means analysts chase false alarms.

• **Recall (TPR)** = TP / (TP + FN). Share of frauds that are caught; low recall means fraud passes through undetected.

• **F1** = 2·P·R / (P + R). Harmonic mean of precision and recall, giving them equal weight.

• **F2** = 5·P·R / (4·P + R). F-beta with beta = 2; weights recall four times as much as precision, matching the higher cost of a missed fraud.

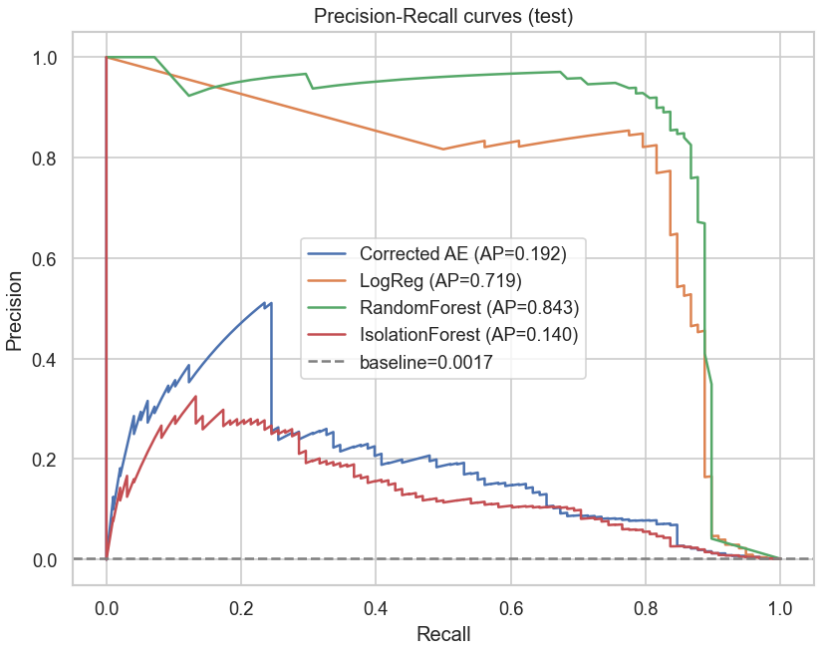
• **MCC** = (TP·TN − FP·FN) / √((TP+FP)(TP+FN)(TN+FP)(TN+FN)). Correlation between predictions and labels using all four confusion-matrix cells; robust under imbalance, with 1 perfect and 0 random.

• **ROC-AUC** = area under TPR vs FPR. Ranking quality across all thresholds; optimistic here because the abundant true negatives dominate it.

• **PR-AUC** = area under precision vs recall. Our primary metric; it ignores the many true negatives and focuses on performance for the rare fraud class.

Model	PR-AUC	ROC-AUC	MCC	F2	Prec.	Recall	TP	FP	FN
Random Forest	0.843	0.947	0.799	0.843	0.729	0.878	86	32	12
Logistic Regression	0.719	0.972	0.740	0.793	0.656	0.837	82	43	16
Original AE (@2.9, leaked)	0.235	0.961	0.212	0.224	0.058	0.806	79	1289	19
Corrected AE (fair)	0.192	0.953	0.311	0.386	0.184	0.531	52	230	46
Isolation Forest	0.140	0.949	0.258	0.314	0.098	0.694	68	624	30

Table 1. Test-set performance, ordered by PR-AUC. The best PR-AUC is Random Forest at 0.843. Logistic Regression reaches 0.719 and Random Forest 0.843, against 0.192 for the corrected autoencoder and 0.140 for the Isolation Forest.



Both supervised baselines reach a higher PR-AUC than the autoencoder, and Random Forest is the strongest model overall. The original autoencoder's ROC-AUC of 0.961 is the figure the tutorial highlights, yet its PR-AUC and precision are the weakest in the table. This is the difference the study set out to measure.

6. Conclusions

Key findings. The source's ROC-AUC reproduces at 0.961, but the figure is inflated by three test-set leaks and is the wrong summary for a 0.17% positive rate. On PR-AUC, MCC and F2, plain supervised models outperform the autoencoder, with Random Forest in front. The published code also does not run without being ported to a current stack.

Strengths and weaknesses of the source. The tutorial is clear and well written, the semi-supervised idea is reasonable for a setting with few labels, and it does show a precision-recall curve. Its weaknesses are the leaky evaluation, an arbitrary threshold, an uninformative accuracy metric, a discarded temporal feature, and the absence of any supervised baseline for comparison.

Lessons learned and future work. The main lesson is that reproducing a headline number is not the same as validating the claim behind it. Useful extensions would be cost-sensitive thresholding tied to the monetary cost of a missed fraud against a blocked card, probability calibration, time-ordered validation instead of a random split, and use of the time signal that the source discards. Where labels exist we would recommend a supervised model, and reserve the autoencoder or Isolation Forest for the fully unlabelled case.

7. Executive Summary

This project reproduces and critically evaluates a widely shared tutorial that detects credit-card fraud with a Keras autoencoder on the ULB dataset of 284,807 transactions, of which 0.17% are fraudulent. The tutorial reports a ROC-AUC of about 0.95. We reproduce this faithfully, obtaining 0.961, after porting the 2017 TensorFlow 1 code to TensorFlow 2 and restoring the dataset through OpenML.

The critical analysis finds the headline both optimistic and incomplete. The evaluation allows the test set to influence feature scaling, model selection, and the choice of the 2.9 threshold, and ROC-AUC is a flattering metric under this level of imbalance. Measured with PR-AUC, MCC and F2, which are the appropriate metrics for rare-event detection, the autoencoder's PR-AUC is only 0.231. After the leaks are removed and baselines are added, Logistic Regression and Random Forest both outperform the autoencoder, and Random Forest is best, with a PR-AUC of 0.843 against 0.192. In operational terms the autoencoder trades a large number of false alarms for each fraud it catches.

The conclusion is that the author's numbers are reproducible but the recommendation is overstated. An autoencoder is not the preferred tool for this problem when labels are available, since a supervised classifier is simpler, faster to train, and more accurate on the metrics that matter for security operations.

8. Summing It Up

• **Problem:** identify the 0.17% of ULB transactions that are fraudulent.

• **Source:** Valkov, 'Credit Card Fraud Detection using Autoencoders in Keras' (curiousily.com; GitHub, ~588 stars).

• **Dataset:** ULB credit-card fraud, 284,807 transactions, 492 frauds (OpenML id 42175).

• **Methodology:** faithful reproduction with the flaws preserved, then a leak-free rerun and supervised baselines, with validation-selected thresholds, PR-AUC, MCC and F2 evaluation, and an error analysis.

• **Main finding:** the ROC-AUC reproduces at 0.961 but is leaked and misleading; supervised baselines win on PR-AUC, with Random Forest best.

• **Were the claims supported:** the numbers reproduce, but the claim of an effective detector is not supported once appropriate metrics are used.

• **Recommended for similar problems:** not the autoencoder when labels are available; use it only in the unlabelled setting, where an Isolation Forest is a stronger baseline on this data.

Final conclusion. Honest, imbalance-aware evaluation reverses an impressive-looking ROC-AUC and shows that a simpler supervised model performs better on the metrics that reflect real detection performance.